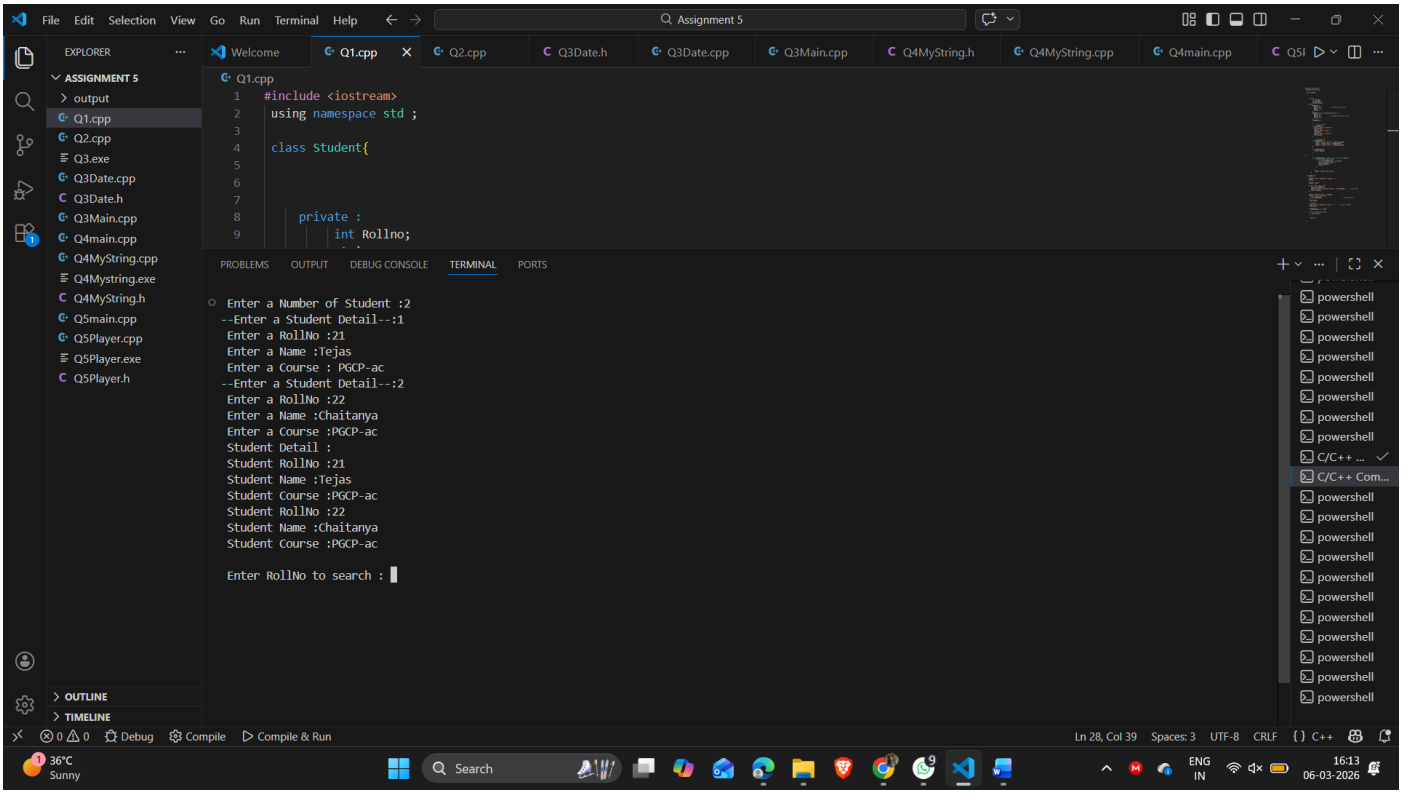
1. **Create Student class with data members RollNo Name Course and implement following • Default constructor • Parameterized constructor • Destructor • Function to accept data • Function for display data • Write main function and create array of students pointer and store data of multiple students in array. • Write Function to find student by Roll No.**

```
#include <iostream>
 using namespace std ;

 class Student{



     private :
          int Rollno;
          string Name ;
          string Course;
     public :
          Student(){
            Rollno = 0;           // Default Constructor
            Name = "";
            Course = "";
          }
          Student(int A, string B,string C )  {
            Rollno = A;
            Name = B;            // Parameterized Constructor
            Course = C;
          }
          ~Student(){

          }
             // Accept data
          void Accept(){
            cout<<" Enter a RollNo :";
            cin>>Rollno;
            cout<<" Enter a Name :";
            cin>>Name;
            cout<<" Enter a Course :";
            cin>>Course;
          }

             // Display data
          void Display(){
              cout<<" Student RollNo :"<<Rollno<<endl;
              cout<<" Student Name :"<<Name<<endl;
              cout<<" Student Course :"<<Course<<endl;
          }

          int getRollno(){
             return Rollno;
          }

};

          void FindStudent( Student *s[], int n, int Roll ){
```

```
                for (int i=0; i<n; i++){
                   if (s[i]->getRollno()  == Roll){
                   cout<<" Student Found ";
                   s[i]->Display();
                   return ;
                }

            }
             cout<<" Student Not Found ";
        }

int main (){
    int n;
    cout<<" Enter a Number of Student :";
    cin>>n;

    Student *s[n];

    for(int i=0; i<n; i++){
       s[i]=new Student();
       cout<<"--Enter a Student Detail--:"<<i+1<<endl;    // Accept Data
       s[i]->Accept();
    }

    cout<<" Student Detail :"<<endl;
     for(int i=0; i<n; i++){
       s[i]->Display();                           // Display Data
     }
     cout<<endl;

     int roll;
     cout<<" Enter RollNo to search : ";     // Search Student
     cin>>roll;

     FindStudent(s, n, roll);

     for(int i=0; i<n; i++){
        delete s[i];
     }

     return 0;

}
```

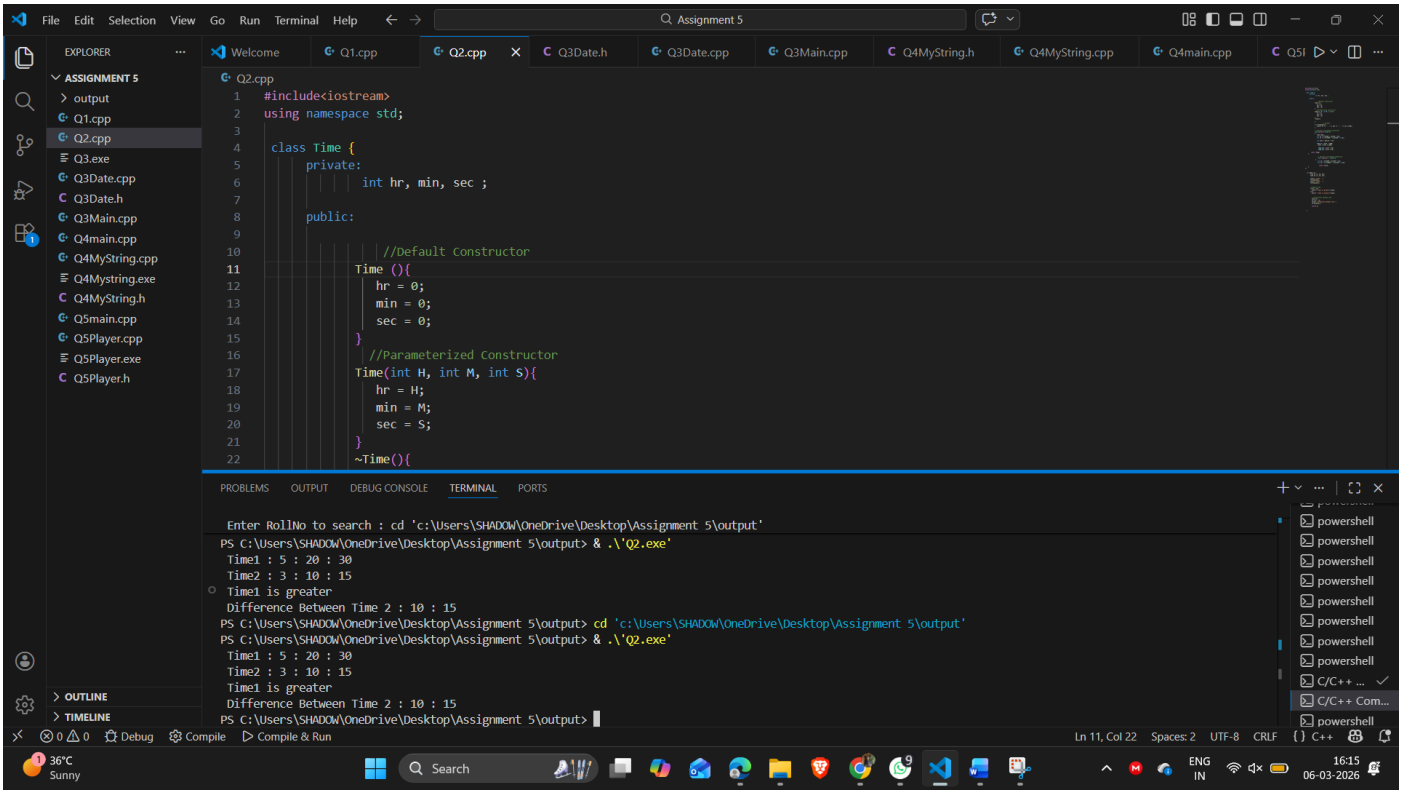


2. Create Time class with data members (use operator overloading) hours minutes seconds and implement following • Default constructor • Parameterized constructor • Destructor • Function for display data • Function to compare two time objects • Function to find out difference between two time objects.

```
#include<iostream>

using namespace std;


class Time {

    private:

        int hr, min, sec ;


    public:


          //Default Constructor

        Time (){

          hr = 0;

          min = 0;

          sec = 0;

        }

         //Parameterized Constructor

        Time(int H, int M, int S){
```

```
        hr = H;
        min = M;
        sec = S;
      }
      ~Time(){

      }
            // Display
      void Display(){
        cout<< hr << " : " << min << " : " << sec <<endl;
      }

      // operator overloading difference
      Time operator-(Time t)
      {
        Time temp ;
        int t1 = hr*3600 + min*60 + sec;
        int t2 = t.hr*3600 + t.min*60 + t.sec;

        int diff = abs(t1 - t2);

        temp.hr = diff / 3600;
         diff = diff % 3600;

         temp.min = diff / 60;
         temp.sec = diff % 60;

   return temp;
}

        // operator overloading comparsion
         bool operator > (Time t)
       {
        int t1 = hr*3600 + min*60 + sec;
```

```
        int t2 = t.hr*3600 + t.min*60 + t.sec;

         return t1>t2;
  }
};


int main (){
    Time t1 (5, 20, 30);
    Time t2 (3, 10, 15);

    cout<<" Time1 : ";
    t1.Display();
    cout<<" Time2 : ";
    t2.Display();

    //Compare Time
    if(t1 > t2)
     cout<<" Time1 is greater"<<endl;
    else
     cout<<" Time2 is greater"<<endl;

     // Differences between time
     Time t3;
     t3 =t1 - t2;
     cout<<" Difference Between Time ";
     t3.Display();

     return 0;


 }
```



3. Create Date class with data members dd mm Assignment No : 5 Submission Date: 6/3/2026 yyyy and implement following • Default constructor • Parameterized constructor • Destructor • Function to find out difference between two dates • Function to compare dates • Separate header and implementation files • Overlaod ==, !=, +, - operator.

Q3Date.h file

```
#ifndef Q3DATE_H

#define Q3DATE_H

#include <iostream>

using namespace std ;

 class Date

 {

  int dd, mm , yyy;


   public:

        Date();

        Date(int , int , int );

        ~Date();


        void display ();


        int operator-(Date);

        bool operator==(Date);
```

```
        bool operator!=(Date);
        Date operator+(int);
 };

 #endif
```

Q3Date.cpp

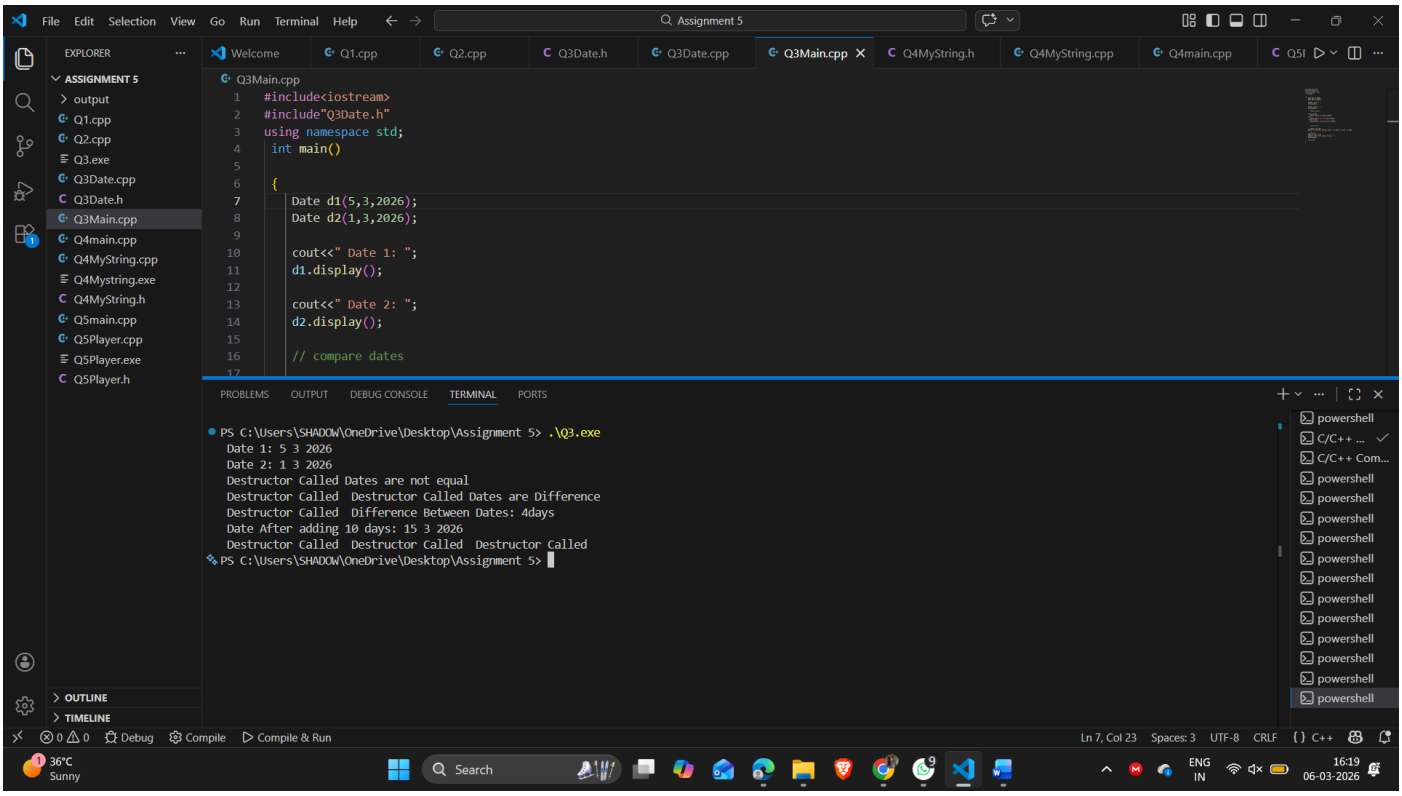
```
#include<iostream>
#include "Q3Date.h"
using namespace std ;
Date::Date()
{
  dd = 1;          // Defalut Constructor
  mm = 1;
  yyy = 2000;
}

Date::Date (int d,int m ,int y)
{
   dd = d;
   mm = m;         // Parameterized Constructor
   yyy = y;
}

Date::~Date()
{
  // Destructor
}

void Date::display()
{
  cout<< dd << " "<< mm << " "<< yyy <<endl;    //display function
}

int Date::operator-(Date d)
```

```
{
   int d1 = dd + mm*30 + yyy*365;          //Difference between two dates
   int d2 = d.dd + d.mm*30 + d.yyy*365;

   return abs(d1-d2);
}

bool Date::operator==(Date d)
{
   return (dd == d.dd && mm == d.mm && yyy == d.yyy);  //Equal operator
}

bool Date::operator!=(Date d)
{                              // Not Equal operator
   return !(*this == d);
}

Date Date::operator+(int d)
{
   Date temp = *this;
   temp.dd += d;

   while( temp.dd > 30)
   {
    temp.dd -= 30;
    temp.mm++ ;

    if(temp.mm > 12){
     temp.mm = 1;
     temp.yyy++;
    }
   }
   return temp;
}
```

Q3main.cpp

```
#include<iostream>
#include"Q3Date.h"
using namespace std;
 int main()

{
  Date d1(5,3,2026);
  Date d2(1,3,2026);

  cout<<" Date 1: ";
  d1.display();

  cout<<" Date 2: ";
  d2.display();

  // compare dates

  if(d1 == d2)
    cout<<"Dates are equal"<<endl;
  else
    cout<<"Dates are not equal"<<endl;
  if(d1 != d2)
    cout<<"Dates are Difference"<<endl;

   // Difference

  int diff = d1 -d2 ;
  cout<<" Difference Between Dates: "<< diff << "days" << endl;

  // Add Days
  Date d3 = d1 + 10;
  cout<<" Date After adding 10 days: ";
  d3.display();
```

```
    return 0;
}
```



4. Create Your own String class (Use class name as MyString) with data members char * p; int size; Allocate memory dynamically for pointer and get size from user. Implement followings • Default constructor • Parameterized constructor • Copy Constructor • Destructor for clean-up (use delete compulsory) • Display function. • Separate header and implementation files.

Q4MyString.h

```
#ifndef Q4MYSTRING_H

#define Q4MYSTRING_H


#include <iostream>

using namespace std;


class MyString

{

    char *p;

    int size;


public:

    MyString();

    MyString(int);
```

```
    MyString(const MyString&);
    ~MyString();

    void display();
};

#endif
```

Q4.MyString.cpp

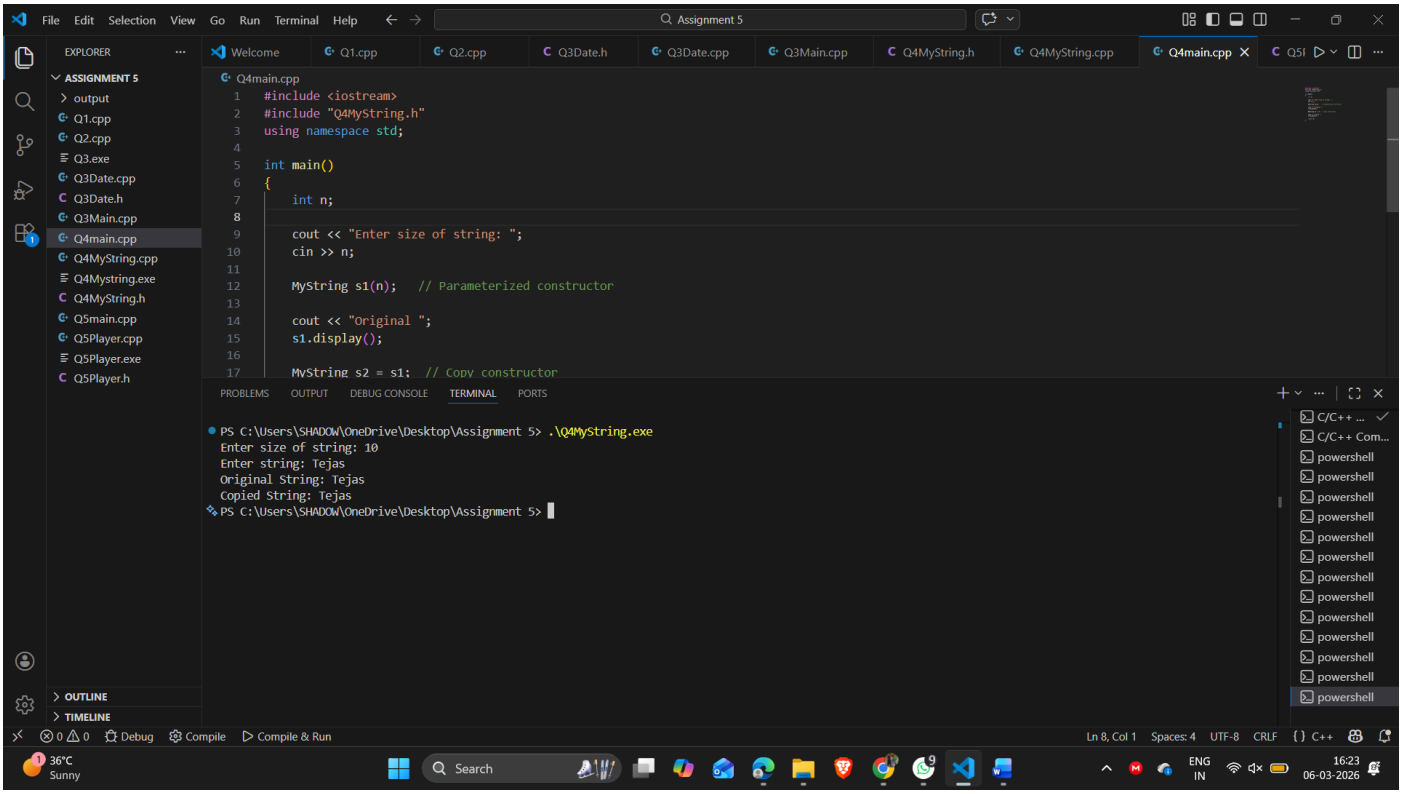
```
#include "Q4MyString.h"

// Default Constructor
MyString::MyString()
{
    size = 0;
    p = NULL;
}

// Parameterized Constructor
MyString::MyString(int s)
{
    size = s;
    p = new char[size + 1];

    cout << "Enter string: ";
    cin >> p;
}

// Copy Constructor
MyString::MyString(const MyString &obj)
{
    size = obj.size;

    p = new char[size + 1];
```

```
    for(int i = 0; i <= size; i++)
    {
        p[i] = obj.p[i];
    }
}

// Destructor
MyString::~MyString()
{
    delete[] p;
}

// Display Function
void MyString::display()
{
    cout << "String: " << p << endl;
}
```

Q4main.cpp

```
#include <iostream>
#include "Q4MyString.h"
using namespace std;

int main()
{
    int n;

    cout << "Enter size of string: ";
    cin >> n;

    MyString s1(n);   // Parameterized constructor

    cout << "Original ";
    s1.display();
```

```
    MyString s2 = s1;  // Copy constructor


    cout << "Copied ";

    s2.display();


    return 0;

}
```



5. Create Player class with data members name age country Implement followings • Default constructor • Parameterized constructor • Function to Sort Array of Player objects (implement as global, sorting should be one name and age) • Accept Function for data input • Display function for data display • Separate header and implementation files • In main function create array of player objects and pass it to Sort function and print sorted player list(sort on name).

Q5Player.h

```
#ifndef Q5PLAYER_H

#define Q5PLAYER_H


#include <iostream>

using namespace std;


class Player

{

    string name;
```

```
    int age;
    string country;

public:
    Player();
    Player(string,int,string);

    void accept();
    void display();

    string getName();
    int getAge();
};

// Global sorting
void sortByName(Player[], int);
void sortByAge(Player[], int);

#endif
```

Q5Player.cpp

```
#include "Q5Player.h"

// Default constructor
Player::Player()
{
    name = "";
    age = 0;
    country = "";
}

// Parameterized constructor
Player::Player(string n, int a, string c)
{
    name = n;
```

```
    age = a;
    country = c;
}

// Accept function
void Player::accept()
{
    cout << "Enter Name: ";
    cin >> name;

    cout << "Enter Age: ";
    cin >> age;

    cout << "Enter Country: ";
    cin >> country;
}

// Display function
void Player::display()
{
    cout << "Name: " << name << endl;
    cout << "Age: " << age << endl;
    cout << "Country: " << country << endl;
    cout << endl;
}

// Getter functions
string Player::getName()
{
    return name;
}

int Player::getAge()
{
```

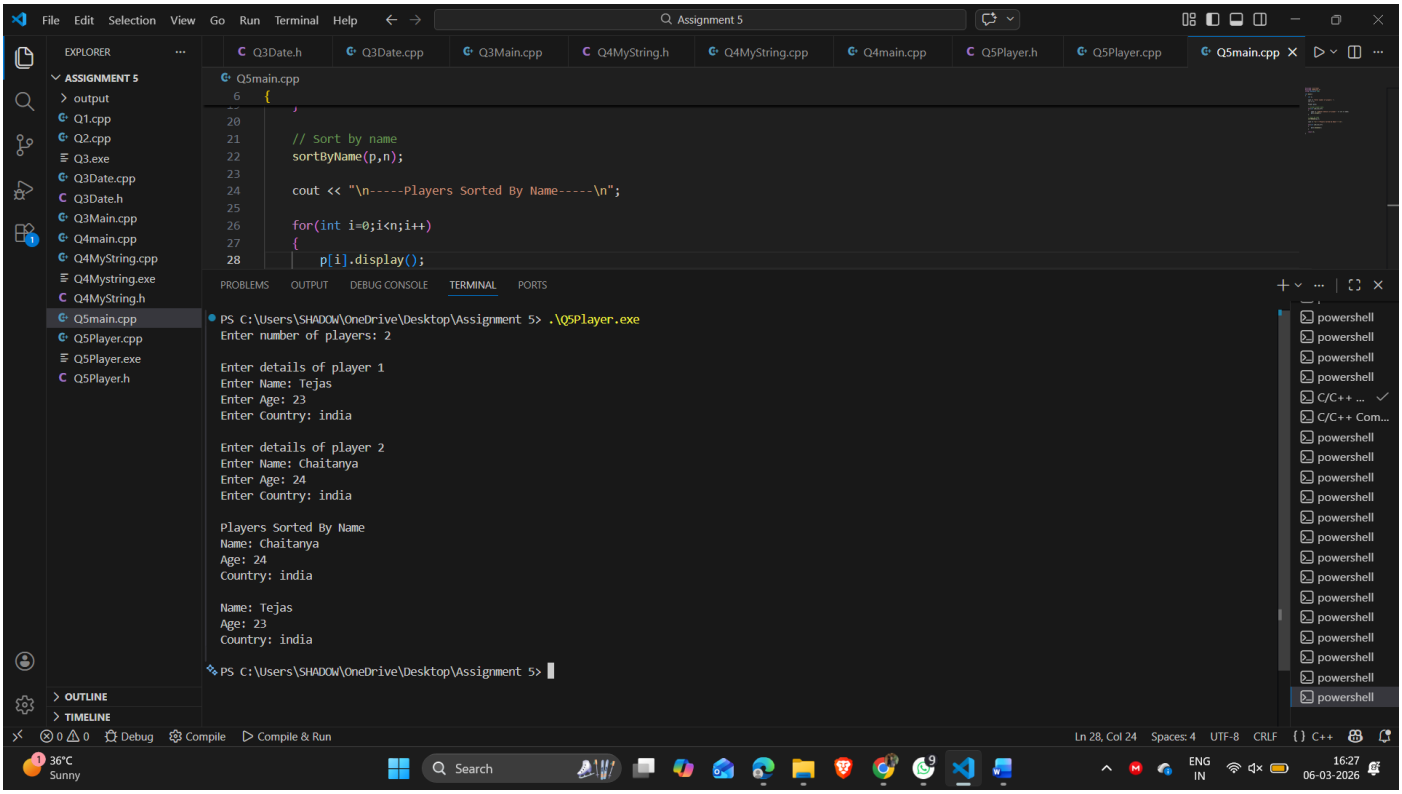
```
    return age;
}

// Sort players by name
void sortByName(Player p[], int n)
{
    for(int i=0;i<n-1;i++)
    {
        for(int j=i+1;j<n;j++)
        {
            if(p[i].getName() > p[j].getName())
            {
                Player temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
}

// Sort players by age
void sortByAge(Player p[], int n)
{
    for(int i=0;i<n-1;i++)
    {
        for(int j=i+1;j<n;j++)
        {
            if(p[i].getAge() > p[j].getAge())
            {
                Player temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
```

```
    }
}
```

Q5main.cpp

```
#include <iostream>
#include "Q5Player.h"
using namespace std;

int main()
{
    int n;

    cout << "Enter number of players: ";
    cin >> n;

    Player p[n];

    // Accept player data
    for(int i=0;i<n;i++)
    {
        cout << "\nEnter details of player " << i+1 << endl;
        p[i].accept();
    }

    // Sort by name
    sortByName(p,n);

    cout << "\n-----Players Sorted By Name-----\n";

    for(int i=0;i<n;i++)
    {
        p[i].display();
    }

    return 0;
```

}


Q5main.cpp
{
// Sort by name
sortByName(p,n);
cout << "\n-----Players Sorted By Name-----\n";
for(int i=0;i<n;i++)
{
p[i].display();
PS C:\Users\SHADOW\OneDrive\Desktop\Assignment 5> .\Q5Player.exe
Enter number of players: 2
Enter details of player 1
Enter Name: Tejas
Enter Age: 23
Enter Country: india
Enter details of player 2
Enter Name: Chaitanya
Enter Age: 24
Enter Country: india
Players Sorted By Name
Name: Chaitanya
Age: 24
Country: india
Name: Tejas
Age: 23
Country: india
PS C:\Users\SHADOW\OneDrive\Desktop\Assignment 5>